

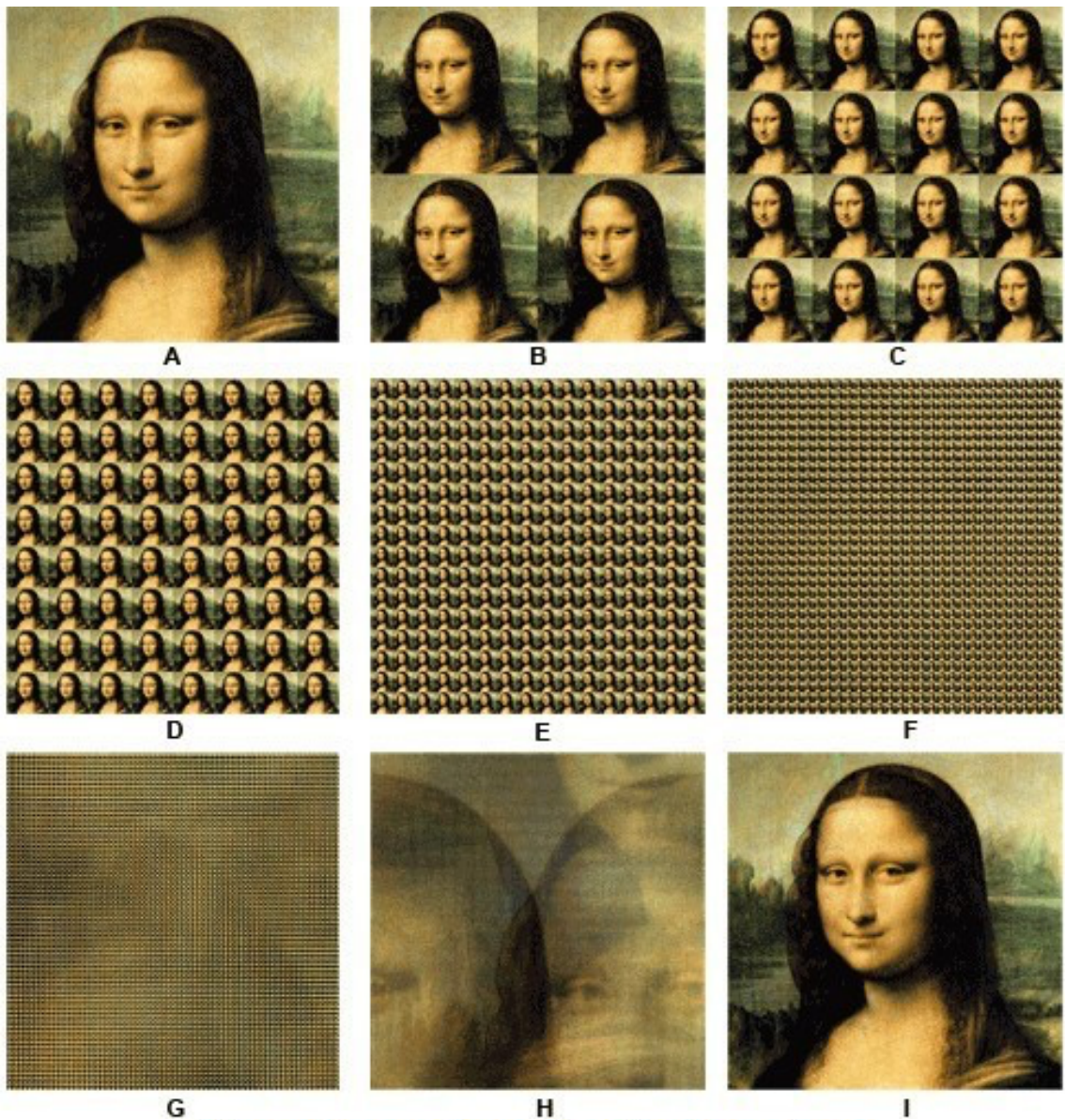
TP 4**1. Le photomaton.**

Dans la série de neuf images A, B, C, D, E, F, G, H, I, chacune est obtenue à partir de la précédente en réduisant l'image d'un facteur 2 dans chaque dimension, ce qui produit quatre sous-images analogues que l'on dispose en carré pour retrouver une image de même taille que l'originale. Le nombre de pixels est exactement conservé : on se contente de déplacer les pixels, sans en modifier la couleur. Plus précisément, on découpe l'image initiale en blocs carrés de quatre pixels (2×2) ; pour chaque bloc, on envoie le pixel en haut à gauche vers la Mona Lisa en haut à gauche, celui en haut à droite vers la Mona Lisa en haut à droite, etc. Cette opération produit bien quatre versions réduites de la Mona Lisa. Cette transformation s'appelle la transformation du photomaton. Ainsi, l'image B comporte 4 Mona Lisa, l'image C en comporte 16, l'image D 64, etc. Fait remarquable : au bout de neuf étapes, l'image de la Mona Lisa réapparaît. Il s'agit bien de la même transformation appliquée itérativement pour obtenir chaque image de la série.

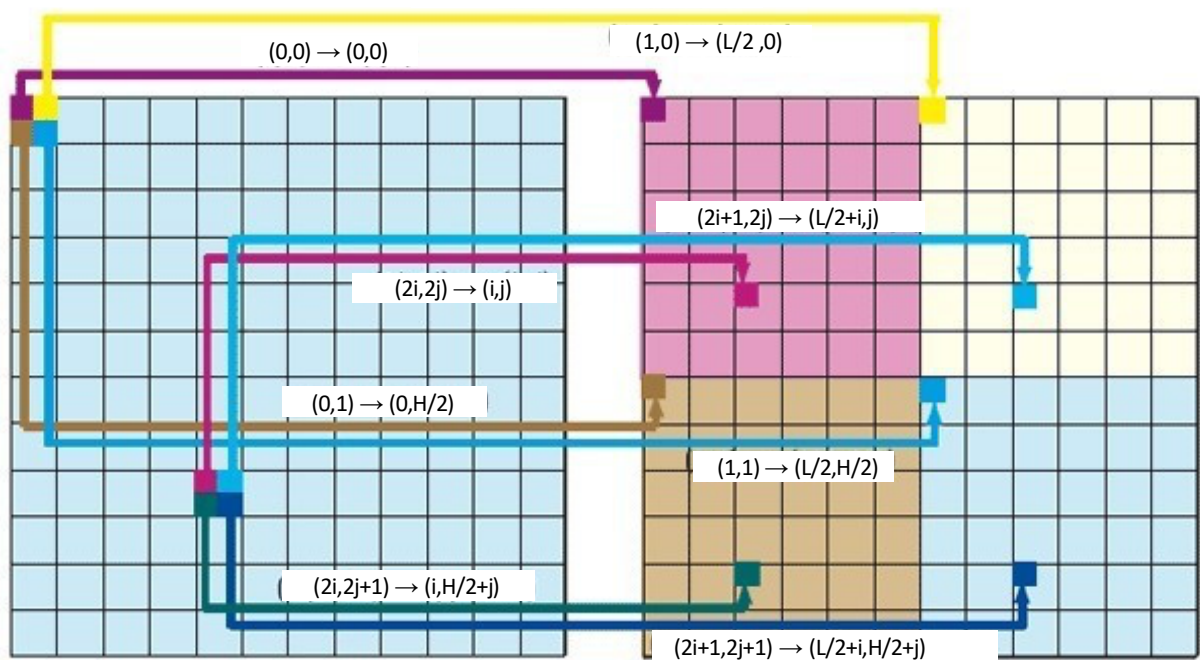
Programmez la transformation du photomaton. Votre programme doit fonctionner pour toutes dimensions paires (nombre pair de lignes et de colonnes). *NB* : on note H la hauteur (axe y) et L la largeur (axe x). L'origine de l'image se situe dans le coin supérieur gauche.

In the series of nine images A, B, C, D, E, F, G, H, I, each one is obtained from the previous by reducing the image by half in each dimension, which produces four analogous sub-images arranged in a square to form an image of the same size as the original. The number of pixels is exactly preserved : we simply rearrange the pixels without altering their color. More precisely, the original image is divided into square blocks of four pixels (2×2); for each block, the top-left pixel is used for the Mona Lisa at the top left, the top-right pixel for the Mona Lisa at the top right, and so on. This operation indeed produces four reduced versions of the Mona Lisa. This transformation is called the photomaton transformation. Thus, image B contains 4 Mona Lisas, image C contains 16, image D contains 64, and so on. Something remarkable happens : after nine steps, the original Mona Lisa image reappears. Note that it is always the same transformation that is applied iteratively to generate each image in the series.

Program the photomaton transformation. Your program must work for any image with even dimensions (an even number of rows and columns). *Note*: H denotes the height (y axis) and L the width (x axis). The origin of the image is located in the upper-left corner.



Transformation du photomaton



2. **All RGB.** Ce projet consiste à transformer une image source en une image transformée contenant une et une fois chaque triplet de couleur RGB. Voici un exemple :

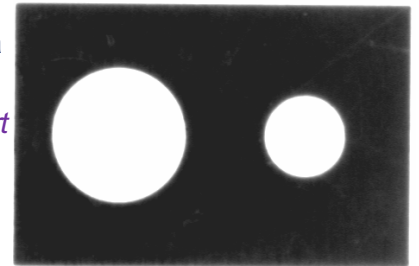
All RGB. This project consists in transforming a source image into a transformed image containing once and once each RGB color triplet. Here is an example :

References : <https://www.youtube.com/watch?v=OuvFsB4SLhA>



3. **Mesures.** Effectuez toutes les mesures nécessaires pour reproduire la pièce représentée dans la figure piece.tif.

Mesures. Perform all the measurements necessary to reproduce the part shown in the figure piece.tif.



4. **Classification de pièces, basée sur l'image.** On se propose de créer un programme (script) permettant de classer les objets de l'image RondelDent en rondelles et en roues dentées en affichant à chaque fois le nombre d'objets de chaque type. Pour ce-faire, il faut respectivement :

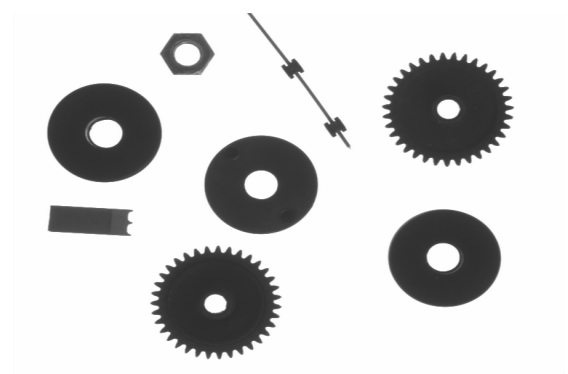
- effectuer des mesures individuelles sur les objets,
- analyser ces mesures,
- filtrer les objets en fonction d'attributs judicieusement choisis,
- afficher conjointement le nombre d'objets de chaque type

Tester le programme sur les deux images test (RondelDent 1 et RondelDent 2)

Image-based object classification. We propose to create a program (script) to classify the objects in the RondelDent image into washers and cogwheels, each time displaying the number of objects of each type. To do this, you must respectively:

- *perform individual measurements on objects,*
- *analyse these measures,*
- *filter objects according to carefully chosen attributes,*
- *jointly display the number of objects of each type*

Test the program on the two test images (RondelDent 1 and RondelDent 2)



5. Deep Learning - CNN : Classification d'images / Image classification

Apprendre comment un CNN reconnaît une image : des filtres appris (convolutions) + pooling pour résumer, et un petit coup d'œil "dans la boîte noire" via feature maps et Grad-CAM.

Le TP est accessible à ce lien et doit être réalisé sur google colab (svp, dupliquez le dossier dans votre espace personnel):

<https://drive.google.com/drive/folders/13cVZBtnPDVPRXR2YeIUxL0-TJfz-F8Vo?usp=sharing>

Objectifs pédagogiques.

- Comprendre le pipeline conv → ReLU → pool → FC.
- Relier filtres appris à des notions de traitement d'image (bords, textures, motifs).
- Démystifier le "black box" avec feature maps + Grad-CAM.
- Lire des métriques de classification (accuracy, matrice de confusion).

Ce qu'on manipule.

- Données : CIFAR-10 (32×32) ou STL-10 (96×96, plus net).
- Modèle : petit CNN à 2 conv + 1-2 couches FC.
- Visualisations : grilles de feature maps (conv1/conv2), Grad-CAM (original/heatmap/overlay).
- Hyperparams "boutons" :
 - nombre de filtres (ex. 6/16/32),
 - taille de noyau (3/5/7),
 - pooling (Max/Avg/ON-OFF).

Déroulé (20-30 min).

1. Entraîner un modèle et afficher accuracy + confusion matrix.
2. Voir les cartes : hooks sur conv1/conv2 → grille de feature maps.
3. Grad-CAM sur 2-3 images : où le modèle "regarde".
4. Varier un seul bouton (filtres, noyau, pooling) → commenter l'effet sur les cartes + accuracy.

Livrable (commentaire dans le code)

1. Expliquer ce qu'est un CNN.
2. 1 grille feature maps (conv1).
3. 2 triptyques Grad-CAM (original / heatmap / overlay).
4. 1 matrice de confusion.
5. 5 lignes d'analyse : effet de l'hyperparamètre modifié (détails/texture, sur/sous-généralisation).

Learn how a CNN recognizes an image: learned filters (convolutions) + pooling to summarize, and a quick peek inside the "black box" via feature maps and Grad-CAM.

The lab is available at this link and must be run on Google Colab (pls. Duplicate the folder to your personal space):

<https://drive.google.com/drive/folders/13cVZBtnPDVPRXR2YeIUxL0-TJfz-F8Vo?usp=sharing>

Learning objectives.

- Understand the pipeline conv → ReLU → pool → FC.
 - Connect learned filters to classic image processing notions (edges, textures, patterns).
 - Demystify the "black box" with feature maps + Grad-CAM.
 - Read classification metrics (accuracy, confusion matrix).

What we'll use.

- Data: CIFAR-10 (32×32) or STL-10 (96×96, sharper).

- Model: a small CNN with 2 conv layers + 1–2 FC layers.
- Visualizations: feature-map grids (conv1/conv2), Grad-CAM (original/heatmap/overlay).
- Hyper-param “knobs”:
 - number of filters (e.g., 6/16/32),
 - kernel size (3/5/7),
 - pooling (Max/Avg/ON-OFF).

Flow (20–30 min).

1. Train a model and display accuracy + confusion matrix.
2. Inspect the maps: add hooks on conv1/conv2 → feature-map grid.
3. Run Grad-CAM on 2–3 images: where does the model “look”?
4. Change one knob (filters, kernel, or pooling) → comment on the effect on the maps + accuracy.

Deliverable (comment in the code)

1. Explain what a CNN is.
2. One feature-map grid (conv1).
3. Two Grad-CAM triptychs (original / heatmap / overlay).
4. One confusion matrix.
5. A 5-line analysis: effect of the modified hyper-parameter (detail/texture, over/under-generalization).

6. Deep Learning – U-net : Segmentation d’images / Image segmentation

Segmenter un objet pixel par pixel avec un U-Net : on compresse pour apprendre *quoi* (encoder), on décompresse pour savoir *où* (decoder) en rebranchant les détails via des *skip connections*, et on évalue avec Dice/IoU — le tout sans ré-entraîner, grâce à des snapshots fournis. Vous êtes libre d’entraîner votre propre réseau en changeant les paramètres si vous le souhaitez (env. 25mn par époque).

Le TP est accessible à ce lien et doit être réalisé sur google colab (svp, dupliquez le dossier dans votre espace personnel) :

https://drive.google.com/drive/folders/1O5CWknFCIMJbJoN-niIyH3IL_dFXen40?usp=drive_link

Objectifs d’apprentissage.

- Comprendre l’architecture U-Net (4 niveaux : encoder → bottleneck → decoder + skips).
- Distinguer perte d’entraînement (BCEWithLogitsLoss) et métriques d’évaluation (Dice/IoU).
- Lire/visualiser des masques : overlays, seuil de décision, FP/FN.
- Conduire une mini-comparaison entre états d’entraînement / seuils / taille d’image.

Ce que vous manipulerez.

- Données : Oxford-IIIT Pet (binaire : {1,2} → 1, 3 → 0).
- Modèle : U-Net 4 niveaux (entrée multiple de 16 : 128/256).
- Snapshots : weights_epoch_{1,3,5,10,15}.pth + best.pt.
- Perte : nn.BCEWithLogitsLoss() (train).
- Métriques : Dice (overlap indulgent) & IoU (plus strict).

Déroulé (20–30 min).

1. Charger & inspecter
 - Charger un snapshot (ex. epoch 10), calculer Dice/IoU (val).
 - Afficher la grille Image / GT / Pred (seuil par défaut THR=0.5).
 - Comparer deux snapshots (ex. epoch 3 vs 10) et commenter (bords, trous, débordements).
2. Seuil de décision
 - Balayer THR ∈ {0.3, 0.5, 0.7} sur la même image.

- Expliquer le compromis FP (déborde sur le fond) / FN (trous dans l'objet).

3. Réglages image/modèle (sans ré-entraîner)

- Taille d'entrée (128 vs 256) avec les mêmes poids → effet sur les bords.
- Overlay FP/FN (rouge/bleu) pour expliciter les erreurs.
- (*Option rapide*) Ablation des skips (démonstration : $\alpha=1$ vs 0) pour montrer leur rôle.

Livrables (commentaire dans le code)

- comparaison de deux snapshots (grilles + Dice/IoU) + 3–4 lignes d'analyse.
- balayage de seuil (0.3/0.5/0.7) sur la même image + commentaire FP/FN.
- 1–2 cas d'échec illustrés et expliqués (zones fines, fond similaire, lumière).

Segment an object pixel by pixel with a U-Net: we compress to learn what (encoder), we decompress to locate where (decoder) while reconnecting details via skip connections, and we evaluate with Dice/IoU—all without retraining, thanks to the provided snapshots. You are free to train your own network by changing parameters if you wish (~25 minutes per epoch).

The lab is available at this link and must be run on Google Colab (please duplicate the folder to your own Drive):

https://drive.google.com/drive/folders/1O5CWknFCIMJbJoN-niIyH3IL_dFXen40?usp=drive_link

Learning objectives.

- Understand the U-Net architecture (4 levels: encoder → bottleneck → decoder + skips).
- Distinguish training loss (BCEWithLogitsLoss) from evaluation metrics (Dice/IoU).
- Read/visualize masks: overlays, decision threshold, FP/FN.
- Run a small comparison across training states / thresholds / input sizes.

What you will work with.

- Data: Oxford-IIIT Pet (binary: $\{1,2\} \rightarrow 1, 3 \rightarrow 0$).
- Model: 4-level U-Net (input size multiple of 16: 128/256).
- Snapshots: weights_epoch_{1,3,5,10,15}.pth + best.pt.
- Loss: nn.BCEWithLogitsLoss() (train).
- Metrics: Dice (more forgiving overlap) & IoU (stricter).

Flow (20–30 min) :

1. Load & inspect
 - Load a snapshot (e.g., epoch 10), compute Dice/IoU (validation).
 - Display the Image / GT / Pred grid (default threshold THR = 0.5).
 - Compare two snapshots (e.g., epoch 3 vs 10) and comment (edges, holes, spillover).
2. Decision threshold
 - Sweep $\text{THR} \in \{0.3, 0.5, 0.7\}$ on the same image.
 - Explain the trade-off FP (bleeds into background) / FN (holes in the object).
3. Image/model settings (no retraining)
 - Input size (128 vs 256) with the same weights → effect on edges.
 - FP/FN overlay (red/blue) to make errors explicit.
 - (Quick option) Skip ablation (démonstration: $\alpha = 1$ vs 0) to show their role.

Deliverables (comment in the code)

- Comparison of two snapshots (grids + Dice/IoU) + 3–4 lines of analysis.
- Threshold sweep (0.3 / 0.5 / 0.7) on the same image + FP/FN commentary.
- 1–2 failure cases illustrated and explained (fine structures, similar background, lighting).